



**ESCUELA TÉCNICA SUPERIOR  
DE  
INGENIERIA INFORMÁTICA**

**Trabajo Fin de Máster**

**Máster en Lógica, Computación, e Inteligencia Artificial**

**Sing2Text: Traducción basada en KeyPoints**

**Realizado por**

Nicolás Rodríguez Ruiz

**Tutorizado por**

Miguel Ángel Martínez Del Amor

---

**Departamento de**

Ciencias de la Computación e Inteligencia Artificial

Sevilla, 22 de junio de 2026







# Índice general

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción</b>                                          | <b>9</b>  |
| 1.1      | Resumen . . . . .                                            | 9         |
| 1.2      | Motivación y Planteamiento . . . . .                         | 9         |
| 1.3      | Objetivo . . . . .                                           | 10        |
| <b>2</b> | <b>Estado del Arte</b>                                       | <b>11</b> |
| 2.1      | General (traducción en SL) . . . . .                         | 11        |
| 2.1.1    | Prediciendo con Glosas . . . . .                             | 11        |
| 2.1.2    | Modelos <i>End2End</i> . . . . .                             | 13        |
| 2.1.3    | LLMS para la traducción de signos . . . . .                  | 13        |
| 2.2      | Uso de keypoints como entrada a modelos en SL . . . . .      | 13        |
| 2.3      | Datasets . . . . .                                           | 13        |
| <b>3</b> | <b>Fundamentos</b>                                           | <b>15</b> |
| 3.1      | Medidas de precisión en traducción . . . . .                 | 15        |
| 3.1.1    | WER . . . . .                                                | 15        |
| 3.1.2    | BLEU . . . . .                                               | 16        |
| 3.1.3    | ROUGE . . . . .                                              | 17        |
| 3.2      | Transformers . . . . .                                       | 18        |
| 3.2.1    | ¿Por qué Transformers? . . . . .                             | 19        |
| 3.2.2    | Mecanismo de Atención . . . . .                              | 19        |
| 3.2.3    | Codificación Posicional . . . . .                            | 21        |
| 3.2.4    | FIXME . . . . .                                              | 22        |
| 3.3      | Clasificación Temporal Conexionista . . . . .                | 22        |
| 3.4      | <i>Decoupled Spatial-Temporal Attention</i> (DSTA) . . . . . | 24        |
| 3.4.1    | Módulos de atención espacial y temporal . . . . .            | 24        |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 3.4.2    | Codificación Posicional . . . . .                    | 26        |
| 3.4.3    | <i>Spatial Global Regularization</i> (SGR) . . . . . | 27        |
| 3.5      | LoRA . . . . .                                       | 29        |
| <b>4</b> | <b>Metodología</b>                                   | <b>31</b> |
| 4.1      | Extracción de Keypoints . . . . .                    | 31        |
| 4.1.1    | Alphapose . . . . .                                  | 32        |
| 4.2      | Normalización de los Datos . . . . .                 | 33        |
| 4.3      | Multi Stream Keypoint Attention . . . . .            | 33        |
| 4.3.1    | Data Augmentation . . . . .                          | 33        |
| 4.3.2    | Tokenizador . . . . .                                | 36        |
| 4.3.3    | Arquitectura del Modelo de Reconocimiento . . . . .  | 36        |
| 4.3.4    | Arquitectura del módulo de traducción . . . . .      | 40        |
| <b>5</b> | <b>Experimentación y análisis de resultados</b>      | <b>43</b> |
| 5.1      | Keypoints . . . . .                                  | 43        |
| 5.2      | MSKA . . . . .                                       | 43        |
| 5.2.1    | Recreando el papel original . . . . .                | 43        |
| 5.2.2    | Utilizando un SLM . . . . .                          | 43        |
| 5.3      | Comparativa con state of the art . . . . .           | 44        |
| <b>6</b> | <b>Conclusión y trabajo futuro</b>                   | <b>45</b> |







# Capítulo 1

---

## Introducción

---

### 1.1. Resumen

### 1.2. Motivación y Planteamiento

Según los datos de la Organización Mundial de la Salud, más de 1,500 millones de personas en el mundo viven con algún grado de pérdida auditiva, de las cuales aproximadamente 430 millones requieren atención especializada. Para una parte significativa de esta población, la lengua de signos es su principal medio de comunicación natural. Sin embargo, el acceso a servicios esenciales como lo son la sanidad, la educación o la administración pública sigue dependiendo casi exclusivamente de intérpretes humanos, un recurso escaso, costoso y con disponibilidad geográfica muy desigual. Esta dependencia limita la autonomía de las personas sordas y perpetúa una brecha de inclusión social difícil de cerrar sin el apoyo de soluciones tecnológicas escalables.

En los últimos años ha habido enormes avances en las técnicas de *Procesamiento del Lenguaje Natural*, *Visión por Computador* y *Aprendizaje Profundo*, así como en la eficiencia y capacidad computacional de los equipos, sin embargo las lenguas de signos han recibido una atención comparativamente menor, y los avances no han alcanzado el nivel de madurez ni la cobertura lingüística que sí presentan los sistemas para lenguas orales. Los sistemas de traducción automática, los asistentes conversacionales y las interfaces de voz han transformado la forma en que las personas oyentes interactúan con la tecnología, pero siguen siendo inaccesibles para quienes se comunican mediante el canal viso-gestual.

Para abordar esta brecha, la investigación en traducción automática de lengua de signos se articula principalmente en torno a dos direcciones complementarias: por un lado, la traducción de texto o voz a lengua de signos, **Text2Sign**, orientada a facilitar la comprensión por parte de personas sordas; por otro, la traducción de lengua de signos a texto, **Sign2Text**, cuyo objetivo es permitir que las personas oyentes o los sistemas automáticos comprendan los mensajes signados. El presente trabajo se centra en esta

segunda dirección, **Sign2Text**, por su impacto directo en la autonomía comunicativa de las personas sordas y por los retos técnicos abiertos que aún presenta.

La tarea **Sign2Text**, también conocida como *Sign Language Recognition and Translation*, persigue la generación automática de texto a partir de una secuencia de signos capturada en vídeo. Un sistema de este tipo permitiría subtítular en tiempo real una conversación signada, dotar a los asistentes virtuales de la capacidad de entender a usuarios sordos, o facilitar el acceso a servicios públicos sin necesidad de un intérprete presente. En definitiva, actuaría como un puente de comunicación directo entre la comunidad sorda y los sistemas digitales diseñados, hasta ahora, pensando exclusivamente en usuarios oyentes.

Desde el punto de vista técnico, Sign2Text es un problema especialmente complejo. A diferencia del reconocimiento de voz, donde la señal de entrada es unidimensional y continua, la lengua de signos es inherentemente multimodal: la información se distribuye simultáneamente entre la configuración y el movimiento de las manos, la expresión facial, la postura corporal y el uso del espacio. Esta riqueza expresiva, que dota a las lenguas de signos de toda su potencia comunicativa, supone al mismo tiempo uno de los mayores desafíos para su modelado automático. A esto se le suma la gran variedad de lenguas de signos que existen, cada una con sus gestos y expresiones visuales únicas, además en la mayoría de los casos, la lengua de signos de una determinada región carece de relación gramatical con la lengua natural de dicha región. Esto dificulta significativamente la comprensión y traducción directa entre un lenguaje natural y de signos.

Además de esta complejidad multimodal, la naturaleza visual de la tarea presenta consideraciones relevantes en materia de privacidad. Procesar vídeos de personas requiere garantías sobre el tratamiento de datos personales, especialmente en entornos sensibles como la sanidad o la educación. Por ello, un enfoque que minimice la exposición de datos desde el origen resulta deseable. A raíz de esto nace la traducción de signos basados en *keypoints*, esto es puntos predeterminados en el cuerpo humano que resumen la información de la postura, expresión e información de manos, capturando la información gestual relevante sin preservar la identidad visual de la persona, permitiendo así un procesamiento más ligero, portable y respetuoso con la privacidad. De forma paralela, el uso de este enfoque, reduce drásticamente la dimensionalidad de los datos, optimizando los costes computacionales de entrenamiento e inferencia y facilitando su despliegue en equipos menos potentes.

### 1.3. Objetivo

# Capítulo 2

---

## Estado del Arte

---

Analizaremos en este capítulo el contexto general de la traducción de lengua de signos, desde su evolución hasta la actualidad. Finalmente estudiaremos el estado del arte específico a los modelos basados en *keypoints* y presentaremos los conjuntos de datos relevantes.

### 2.1. General (traducción en SL)

Como vimos en la introducción, la traducción automática de la lengua de signos es un problema complejo que implica mapear una secuencia de vídeo continuo a una secuencia de texto en un lenguaje hablado, superando grandes barreras espaciales, sintácticas y gramaticales. A lo largo de los últimos años ha habido grandes avances en las técnicas utilizadas. De similar forma a las tareas de traducción tradicionales se han preferido arquitecturas basadas en mecanismos de atención, *vision transformers* o incluso *LLM*.

#### 2.1.1. Prediciendo con Glosas

Muchos de los intentos más exitosos y el paradigma hace unos años era utilizar las glosas como representación intermedia para la traducción. Este enfoque de dos etapas simplificaba el problema, pero sufría del cuello de botella de la propagación de errores. Esto es, cuando la parte del modelo encargada de traducir las glosas comete un error, la segunda parte, por muy buena que sea, al tener información equivocada cometerá un error en la traducción.

#### STMC (Spatial-Temporal Multi-Cue, SLR)

Aunque la lingüística ya reconocía el carácter fundamentalmente multimodal de la lengua de signos, los modelos STMC (Spatial-Temporal Multi-Cue) fueron pioneros en

integrar esta premisa directamente en el diseño arquitectónico de una red neuronal entrenable de principio a fin. A diferencia de enfoques anteriores que dependían de algoritmos externos (como rastreadores de manos) o que simplemente concatenaban características perdiendo información, las redes **STMC**, propuestas por Zhou et al. [7] en 2020, procesan de forma conjunta diferentes señales visuales (forma de las manos, expresiones faciales, postura corporal) en dimensiones espaciales y temporales para mejorar el reconocimiento de glosas. Para lograr esto, la arquitectura del modelo se divide en dos componentes principales:

- **Módulo Espacial Multiseñal (*Spatial Multi-Cue*)** que se encarga de la representación espacial en cada fotograma. Incorpora una rama de estimación de pose integrada y diferenciable que permite recortar y extraer automáticamente las características visuales específicas de cada canal.
- **Módulo Temporal Multiseñal (*Temporal Multi-Cue*)** que modela las dependencias temporales a lo largo del video a través de dos rutas paralelas. La ruta intra-señal preserva las características únicas y la evolución temporal de cada canal de forma aislada, mientras que la ruta inter-señal explora la sinergia y colaboración entre todas las señales a diferentes escalas de tiempo.

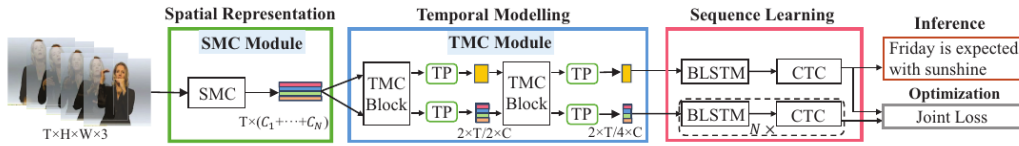


Figura 2.1: Estructura del modelo STMC

### Spotter+GPT (Traducción Híbrida de SLT)

Aunque la traducción de la lengua de signos se ha formulado frecuentemente como una tarea de traducción automática neuronal compleja debido a los desafíos para alinear y tokenizar videos, además de las marcadas diferencias gramaticales entre lenguajes hablados y de signos, el enfoque propuesto por Sincan et al. [10] en 2024 innova al prescindir del entrenamiento de un modelo de traducción glosa-a-texto. A diferencia de los enfoques convencionales que pueden presentar un bajo rendimiento en dominios extensos, el método **Spotter+GPT** propone una arquitectura híbrida que combina el reconocimiento visual clásico con las capacidades de razonamiento de un **LLM** preentrenado. Para lograr esto, la arquitectura del sistema se divide en dos componentes principales:

- **Módulo de Detección Visual (*Sign Spotter*)** que utiliza un modelo I3D entrenado con un gran conjunto de datos de dominio lingüístico para identificar signos individuales. Este módulo procesa el video continuo mediante una técnica de ventana deslizante para extraer predicciones de glosas, aplicando posteriormente un filtrado basado en umbrales de confianza para consolidar la secuencia.

- **Módulo de Generación de Lenguaje (*ChatGPT* / *LLM*)** que emplea la versión *gpt-3.5-turbo* para recibir la secuencia de glosas detectadas de forma directa. Mediante ingeniería de instrucciones (*prompting*) y sin requerir ajuste fino (*fine-tuning*), el modelo transforma esta lista en una oración coherente en lenguaje hablado, superando de forma dinámica las diferencias en el orden gramatical.

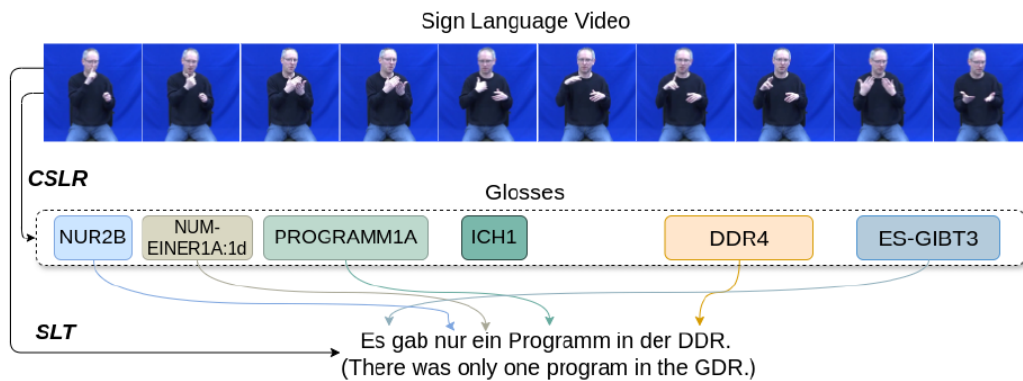


Figura 2.2: Estructura del modelo spotter+GPT

En cuanto a los resultados, los experimentos muestran que el Spotter+GPT supera a modelos secuenciales dedicados en métricas de calidad humana y similitud semántica (como BLEURT). Además, destaca cualitativamente por su gran resiliencia, logrando mantener la coherencia semántica en la oración hablada final incluso cuando el detector visual pierde información o introduce glosas adicionales.

- Transformers

### 2.1.2. Modelos *End2End*

- Gloss-Free SLT
- Sign2GPT (El Paradigma Gloss-based con LLM)

### 2.1.3. LLMS para la traducción de signos

## 2.2. Uso de keypoints como entrada a modelos en SL

## 2.3. Datasets

El paradigma en cuanto conjuntos de datos no es muy diferente al de hace unos años. Esto radica principalmente en la dificultad y el coste que existe en la traducción manual de miles de videos de lengua de signos.



# Capítulo 3

---

## Fundamentos

---

### 3.1. Medidas de precisión en traducción

A diferencia de las tareas de aprendizaje automático tradicional, como la clasificación o la regresión, donde el rendimiento puede medirse de forma exacta mediante métricas como el accuracy o el error cuadrático medio, la traducción de lenguaje natural carece de una única respuesta correcta.

El lenguaje humano es inherentemente complejo, subjetivo y altamente combinatorio. Ante una tarea de traducción, resumen o generación de texto, existen múltiples formas válidas de expresar la misma idea utilizando diferentes estructuras sintácticas o sinónimos. Esta flexibilidad hace que comparar la salida de un modelo algorítmico con un texto de referencia sea un problema no trivial.

Históricamente, el estándar de oro para evaluar la calidad del texto generado ha sido la evaluación humana. Contar con lingüistas o hablantes nativos que juzguen la fluidez, la coherencia y la adecuación de un texto garantiza resultados fiables. No obstante, la evaluación manual es un proceso costoso, lento y difícilmente escalable. En el contexto actual, donde los modelos de lenguaje requieren iteraciones constantes y pruebas sobre conjuntos de datos masivos, depender exclusivamente del juicio humano resulta inviable.

En este contexto surgen, entre otras, las medidas aquí expuestas, buscando obtener la mayor correlación con el criterio de un evaluador humano.

#### 3.1.1. WER

La métrica **Word Error Rate (WER)** tiene su origen en la evaluación de los primeros sistemas de **Reconocimiento Automático de Habla (ASR)**, por sus siglas en inglés) desarrollados entre las décadas de 1980 y 1990. Ante la necesidad de cuantificar el rendimiento de los modelos ocultos de Markov (*HMM*) en la transcripción de voz a texto, la comunidad científica adoptó y adaptó la **distancia de Levenshtein**. Mientras

que Levenshtein medía la distancia de edición a nivel de caracteres, el WER elevó esta abstracción al nivel de palabras (o tokens), convirtiéndose en la métrica estándar para medir la robustez de las secuencias decodificadas frente a una transcripción de referencia.

### Funcionamiento

El WER se define como la distancia de edición mínima (a nivel de palabra) requerida para alinear la secuencia producida por el modelo con la secuencia de referencia (la transcripción real o *ground truth*), normalizada por la longitud de la referencia.

Matemáticamente, se basa en la suma de tres tipos de errores de edición. Si denotamos la secuencia de referencia como  $R$  y la secuencia de hipótesis (la salida del modelo) como  $H$ , el cálculo se realiza mediante programación dinámica para encontrar la alineación óptima que minimice la siguiente función.

$$WER = \frac{S + D + I}{N},$$

donde,

- **S** es el número de palabras en la referencia que han sido reemplazadas por una palabra incorrecta en la hipótesis,
- **D** es el número de palabras presentes en la referencia que el modelo ha omitido,
- **I** es el número de palabras añadidas por el modelo que no existen en la referencia,
- **N** es el número total de palabras en la secuencia de referencia.

A pesar de su adopción generalizada, el WER presenta grandes deficiencias al evaluar lenguajes humanos. Su principal limitación es la absoluta ausencia de comprensión semántica, ya que penaliza por igual errores menores (como omitir una partícula) y sustituciones que alteran completamente el significado de la frase. Además, resulta hipersensible al reordenamiento y carece de un límite superior acotado, permitiendo valores por encima del 100 % que dificultan su interpretación intuitiva frente a otras métricas de precisión.

A pesar de estas limitaciones, el WER es la métrica indispensable en reconocimiento de signos por su naturaleza secuencial y por exigencia empírica. A nivel teórico, la extracción de glosas es un problema de alineamiento temporal similar al reconocimiento automático de voz, donde esa hipersensibilidad a ligeros cambios en el orden de las glosas no es un defecto, ya que una glosa fuera de tiempo indica un error real de segmentación del modelo.

### 3.1.2. BLEU

Propuesta en 2002, **BLEU** (*BiLingual Evaluation Understudy*) [13] es una de las métricas más utilizadas para evaluar la calidad de una traducción automática. Su premisa fundamental consiste en medir la similitud entre el texto generado por un modelo y una o varias traducciones humanas de referencia. Para ello, BLEU no evalúa palabras aisladas, sino secuencias de  $n$  palabras, conocidas como  $n$ -gramas. Esto permite cuantificar



simultáneamente la adecuación (si se usa el vocabulario correcto) y la fluidez (si el orden y la coherencia son adecuados).

Para evitar que un modelo obtenga una puntuación artificialmente alta repitiendo una misma palabra correcta, BLEU utiliza una precisión modificada o “recortada” (*clipped*). Esto significa que el número de veces que se contabiliza un acierto para un  $n$ -grama en la traducción generada se limita al número máximo total de veces que dicho  $n$ -grama aparece en la referencia.

BLEU es una métrica a nivel de **corpus**, el cálculo no se promedia independientemente frase por frase, sino que se suman los aciertos recortados de todas las oraciones y se dividen por el número total de  $n$ -gramas generados en todo el texto candidato.

$$p_n = \frac{\sum_{C \in \{\text{Candidatos}\}} \sum_{n\text{-grama} \in C} \text{Count}_{\text{clip}}(n\text{-grama})}{\sum_{C' \in \{\text{Candidatos}\}} \sum_{n\text{-grama}' \in C'} \text{Count}(n\text{-grama}')}$$

Sin embargo, basarse únicamente en esta precisión presenta un sesgo ya que las traducciones extremadamente cortas podrían obtener puntuaciones perfectas omitiendo información difícil de traducir. Para contrarrestar esta tendencia, la métrica introduce la penalización por brevedad  $BP$ . Esta penalización reduce la puntuación final si la longitud total  $c$  de las traducciones generadas es menor o igual a la longitud total  $r$  del corpus de referencia.

$$BP = \begin{cases} 1 & \text{si } c > r \\ e^{1-\frac{r}{c}} & \text{si } c \leq r \end{cases}$$

Finalmente, la puntuación total de BLEU se obtiene combinando la penalización por brevedad con las precisiones de los  $n$ -gramas, ponderadas por pesos uniformes  $w_n = 1/N$

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log P_n\right)$$

El resultado final de BLEU es un valor entre 0 y 1 (o 0 y 100), donde un valor más cercano a 1 (o 100) indica una mayor calidad global de la traducción.

Al basarse estrictamente en la coincidencia léxica exacta, no comprende sinónimos ni parafraseos. Una traducción perfectamente válida que utilice vocabulario diferente al de la referencia obtendrá una puntuación BLEU muy baja. Además, no tiene en cuenta la estructura gramatical o el significado global de la frase.

Implementación de [11].

### 3.1.3. ROUGE

Introducida en 2004, **ROUGE** (*Recall-Oriented Understudy for Gisting Evaluation*) [14]. Su formulación matemática se centra en la exhaustividad (*recall*). El objetivo principal es verificar cuánta de la información crucial, presente en los resúmenes de referencia humanos, ha sido capturada por el modelo.

Aunque ROUGE es una familia de métricas, las formulaciones más relevantes se dividen en dos enfoques principales: el conteo de  $n$ -gramas (ROUGE-N) y la subsecuencia común más larga (ROUGE-L).

**ROUGE-N** mide el solapamiento de  $n$ -gramas entre el resumen candidato (generado por la máquina) y los resúmenes de referencia. Se calcula con la siguiente fórmula.

$$\text{ROUGE-N}_{\text{Recall}} = \frac{\sum_{S \in \{\text{Referencias}\}} \sum_{n\text{-grama} \in S} \text{Count}_{\text{match}}(n\text{-grama})}{\sum_{S \in \{\text{Referencias}\}} \sum_{n\text{-grama} \in S} \text{Count}(n\text{-grama})},$$

donde  $\text{Count}_{\text{match}}(n\text{-grama})$  es el número máximo de  $n$ -gramas que coocurren tanto en el resumen candidato como en el conjunto de resúmenes de referencia, y el denominador es el número total de  $n$ -gramas presentes en las referencias.

Sin embargo, para evitar que un modelo genere un texto excesivamente largo que contenga todas las palabras del diccionario (lo que daría una exhaustividad del 100%), las implementaciones modernas de ROUGE también calculan la **precisión**, cambiando el denominador para contar los  $n$ -gramas del texto candidato.

$$\text{ROUGE-N}_{\text{Precision}} = \frac{\sum_{S \in \{\text{Referencias}\}} \sum_{n\text{-grama} \in S} \text{Count}_{\text{match}}(n\text{-grama})}{\sum_{n\text{-grama} \in \text{Candidato}} \text{Count}(n\text{-grama})}$$

Finalmente, se combinan ambas métricas utilizando el  $F_1$ -score (la media armónica).

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Por su parte, **ROUGE-L** evalúa la similitud entre textos mediante la **subsecuencia común más larga** (LCS). Para entender qué es una subsecuencia, conviene distinguirla de una subcadena, mientras que una subcadena exige que las palabras sean consecutivas, una **subsecuencia** solo exige que aparezcan en el mismo orden relativo, aunque haya otras palabras intercaladas.

BLEU castiga que el modelo invente texto mediante la precisión, y utiliza la penalización por brevedad para evitar textos excesivamente cortos. ROUGE, en cambio, castiga que el modelo omita información clave mediante la exhaustividad, y confía en la puntuación  $F_1$  para penalizar indirectamente a los modelos que generan textos excesivamente largos.

Al igual que BLEU, ROUGE sufre de la misma limitación semántica, al basarse en solapamientos léxicos directos, no es capaz de puntuar correctamente el uso de sinónimos o paráfrasis.

### 3.2. Transformers

La aparición de la arquitectura **transformer**, introducida por Vaswani et al. [3], supuso un cambio de paradigma en el procesamiento de secuencias, desplazando a las redes recurrentes en tareas de modelado de lenguaje. La relevancia de este modelo en el presente

trabajo radica en su capacidad para gestionar dependencias de largo alcance mediante el mecanismo de auto-atención (*self-attention*), el cual permite que cada elemento de una secuencia (ya sea una palabra o un *keypoint* del cuerpo) sea procesado en relación con todos los demás de forma paralela. Por ello, dedicaremos esta sección a entender el funcionamiento de los *transformers* con cierto nivel de detalle.

### 3.2.1. ¿Por qué Transformers?

Para entender la relevancia e innovación de los transformers es necesario conocer lo que les precedía. En particular, las *long short-term memory* y las *gated recurrent neural networks* dominaron el procesamiento de secuencias mediante mecanismos de computación recurrente, los cuales procesan la información de manera secuencial siguiendo el orden temporal de la entrada. Estas arquitecturas presentaban dos limitaciones fundamentales: la dificultad para captar dependencias de largo alcance debido al desvanecimiento del gradiente y la imposibilidad de paralelizar el entrenamiento, lo que restringe enormemente la eficiencia computacional con secuencias largas.

Los transformers proponen abandonar por completo la recurrencia en favor del mecanismo de atención, lo que les permite captar dependencias de largo alcance en los textos de forma directa y con operaciones fácilmente paralelizables. En lugar de procesar la secuencia elemento a elemento, estos modelos consideran simultáneamente todos los tokens de entrada mediante el mecanismo de auto-atención, que calcula la relevancia relativa entre cada par de posiciones de la secuencia.

Este enfoque introduce varias ventajas clave. En primer lugar, elimina la dependencia estrictamente secuencial del cómputo, permitiendo un uso mucho más eficiente del hardware moderno, especialmente en arquitecturas paralelas como las **GPUs**. En segundo lugar, facilita el modelado de relaciones a larga distancia sin los problemas asociados al desvanecimiento del gradiente, ya que cualquier elemento de la secuencia puede interactuar directamente con cualquier otro en un solo paso de atención.

Además, los transformers incorporan mecanismos como las codificaciones posicionales para preservar la información sobre el orden de la secuencia, compensando así la ausencia de recurrencia. Esta combinación de atención global y representación posicional ha demostrado ser altamente efectiva en tareas de modelado del lenguaje y traducción automática, superando ampliamente a las arquitecturas recurrentes tradicionales.

Es natural plantearse si esta habilidad y facilidad que presentan los transformers en la traducción de lenguajes se translada a la lengua de signos utilizando los *keypoints* como representación de los signos.

### 3.2.2. Mecanismo de Atención

La función de atención es un mecanismo que permite a un modelo identificar y priorizar la información más relevante dentro de un conjunto de datos. Para ello, compara una consulta, *query* con un conjunto de claves, *keys*, mediante una función de compatibilidad,

que mide qué tan bien coincide la *query* con cada *key*, generando pesos que reflejan la importancia de cada elemento. Transformando estos pesos en una distribución de probabilidad mediante la función *softmax*, se calcula una combinación ponderada de los valores *values*, produciendo así una representación que enfatiza los componentes más pertinentes.

La función de atención más relevante es la *Scaled Dot-Product Attention* cuya entrada son claves y consultas de dimensión  $d_k$  y valores de dimensión  $d_v$ . En la práctica, se saca provecho de que la multiplicación entre matrices está muy optimizada.

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK'}{\sqrt{d_k}} \right) V.$$

Se añade el factor de escalado  $\sqrt{d_k}$ , pues si las matrices de *query* y *key* son de una muy alta dimensión, es posible que los valores exploten en tamaño, donde la función *softmax* se vuelve muy “extrema”, es decir, asigna casi todo el peso a un solo elemento y hace que los gradientes sean muy pequeños, dificultando el entrenamiento.

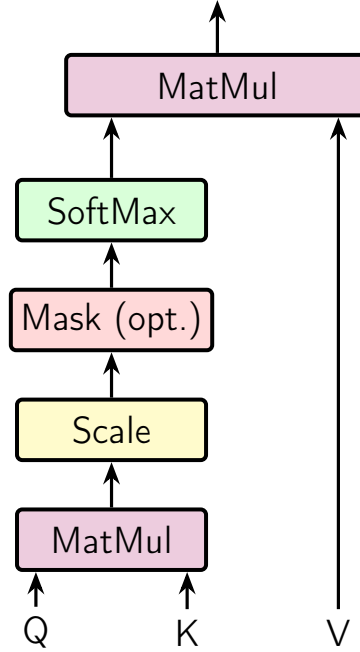


Figura 3.1: Una cabeza de atención

La idea detrás de los mecanismos de atención es, como se menciona, enriquecer el significado de cada uno de los *tokens* teniendo en cuenta los otros de la consulta. Esto permite que el modelo tenga la información más completa posible a la hora de realizar la traducción. Un problema que surge es la limitada capacidad de una sola cabeza de prestar atención a múltiples aspectos de la entrada, pues al tratar de hacerlo se diluyen las relaciones al mezclarse, potencialmente limitando la capacidad de comprensión del modelo.

Así, en vez de tener una única función de atención con *keys*, *values* y *queries*  $d_{\text{modelo}}$ -dimensionales, **Vaswani et al.** proponen utilizar  $h$  diferentes proyecciones lineales a di-

mensión  $d_k$ , para consultas y claves, y  $d_v$  para valores. De esta forma el modelo puede “prestar atención” a distintos aspectos de la entrada de forma simultánea.

En este caso, la atención multi-cabeza se calcula como sigue

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_h)W^O$$

con  $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$ ,

donde  $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{modelo}} \times d_k}$ ,  $W_i^V \in \mathbb{R}^{d_{\text{modelo}} \times d_v}$  y  $W^O \in \mathbb{R}^{hd_v \times d_{\text{modelo}}}$  son matrices de pesos entrenables. En particular las matrices  $W$  son las proyecciones.

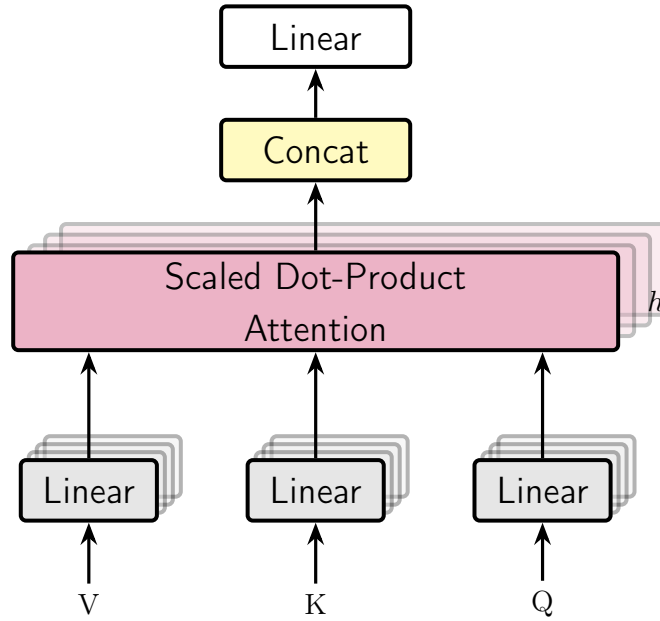


Figura 3.2: Diagrama de la atención multi-cabeza

### 3.2.3. Codificación Posicional

$$PE(p, 2i) = \sin(p/10000^{2i/C_{in}}) \quad (3.1)$$

$$PE(p, 2i + 1) = \cos(p/10000^{2i/C_{in}}) \quad (3.2)$$

Donde,

- $p$  es la posición del elemento en la secuencia.
- $i$  es la dimensión del vector de codificación.
- $C_{in}$  es el número de canales de entrada.

### 3.2.4. FIXME

Las funciones de codificación utilizan ondas sinusoidales y cosenoidales de diferentes frecuencias. Denotando  $p$  a la posición de la articulación en la secuencia,  $i$  a la dimensión específica del vector de codificación posicional y  $C_{in}$  la profundidad del canal de entrada la codificación posicional se calcula mediante las siguientes fórmulas.

$$PE(p, 2i) = \sin(p/10000^{2i/C_{in}})$$

$$PE(p, 2i + 1) = \cos(p/10000^{2i/C_{in}})$$

La naturaleza periódica proporciona diferentes representaciones para cada posición, lo que permite al modelo comprender mejor las relaciones posicionales relativas entre los elementos de la secuencia. Las articulaciones dentro de un mismo fotograma se codifican secuencialmente, mientras que las articulaciones idénticas en diferentes fotogramas comparten una codificación común.

## 3.3. Clasificación Temporal Conexionista

La *Connectionist Temporal Classification* es un método que surge para entrenar Redes Neuronales Recurrentes en el etiquetado de secuencias continuas, eliminando la necesidad de utilizar datos de entrenamiento presegmentados o alineados manualmente.

Esto es especialmente importante en tareas perceptuales y de visión por computador, donde la secuencia de entrada, por ejemplo, una serie de fotogramas de vídeo que capturan a un signante, es habitualmente más larga que la secuencia objetivo de glosas o etiquetas a predecir. Al ser de longitudes distintas y carecer de una segmentación temporal explícita, no existe una correspondencia o alineación a priori entre ambas secuencias.

Debido a la alta carga matemática de este concepto, nos limitaremos a explicar el funcionamiento y la idea que propone a grandes rasgos sin entrar en la carga teórica pues no es relevante a este trabajo, aún así todo el formalismo teórico se encuentra en *Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks* [15].

Para resolver este problema de alineación, la intuición principal de la CTC radica en interpretar las salidas de la red neuronal (la secuencia completa) como una distribución de probabilidad sobre todas las posibles **secuencias de etiquetas**, condicionadas a la secuencia de entrada. Combinan dos técnicas principalmente.

### *Blank Label*

Para cada fotograma individual (cada instante de tiempo), a la capa softmax que nos devolvía una distribución de probabilidad sobre todas las glosas posibles del vocabulario, se extiende con un token especial “BLANK”. Este token representa la probabilidad de no observar ninguna etiqueta en ese instante de tiempo específico.

### Mapping

La red genera predicciones para cada fotograma individual, pero la CTC aplica una regla de transformación para convertir esa ruta temporal extendida en una secuencia final coherente. Esta regla consiste en eliminar primero todas las etiquetas que se repiten de forma consecutiva y, posteriormente, suprimir todos los tokens en blanco.

De forma intuitiva, esto significa que el modelo registrará una nueva etiqueta únicamente cuando la red pase de predecir un espacio en blanco a predecir una glosa, o cuando cambie directamente de una glosa a otra distinta.

Llevado a la traducción de lengua de signos, si un gesto dura varios fotogramas, una idea del proceso es la siguiente.

$$\underbrace{[-, -, \text{HOLA}, \text{HOLA}, \text{HOLA}, -, -]}_{7 \text{ fotogramas}} \rightarrow \underbrace{[\text{HOLA}]}_{\text{Una sola glosa}}$$

Finalmente, durante la fase de entrenamiento, la CTC no penaliza a la red por no adivinar el momento exacto en el que empieza o termina un signo. En su lugar, CTC le da ignora exactamente cuándo ocurre el signo, acepta que hay muchas formas válidas de acertar a lo largo del tiempo. Para un vídeo de 5 frames, todas estas “rutas” de la red neuronal son válidas porque, al aplicar la regla de la CTC, todas colapsan en el mismo resultado final, [HOLA].

- **Ruta A:** [GRACIAS, GRACIAS, -, -, -]
- **Ruta B:** [-, GRACIAS, GRACIAS, -, -]
- **Ruta C:** [-, -, -, GRACIAS, GRACIAS]

Durante el entrenamiento, no se intenta forzar que ocurra solo una ruta que se considere “perfecta”. Lo que se hace es sumar la probabilidad de cada ruta posible que devuelva el mismo resultado tras aplicar las reglas. Al maximizar esta suma, la red se entrena para predecir la etiqueta correcta sin atarse a una plantilla de tiempo rígida.

En su lugar, utiliza una función objetivo (CTCloss) que suma las probabilidades de todas las alineaciones válidas posibles, maximizando de forma directa la probabilidad global de obtener la secuencia de glosas correcta. Esto permite que la red aprenda automáticamente de forma discriminativa y se vuelva robusta ante variaciones en la velocidad o duración temporal de los signos.

Si  $x$  representa la secuencia de entrada,  $z$  es la secuencia objetivo, y  $p(z|x)$  es la probabilidad total acumulada de todas las alineaciones válidas que colapsan en esa secuencia de glosas tras aplicar las reglas de remoción de blancos y repetidos. La función de pérdida CTC se define como la verosimilitud logarítmica negativa de las secuencias objetivo mapeadas.

$$O^{ML}(S, N_w) = - \sum_{(x,z) \in S} \log p(z|x)$$

### 3.4. Decoupled Spatial-Temporal Attention (DSTA)

Tradicionalmente, los métodos de reconocimiento de acciones (basados en redes *RNN* o *CNN*) dependían de reglas manuales o topologías de grafos prediseñadas para modelar cómo se conectan las articulaciones del cuerpo. El problema es que estos diseños manuales no siempre son óptimos y limitan la generalización del modelo.

Para solucionar esto, **Lei Shi et al.** en [12] proponen un enfoque basado exclusivamente en mecanismos de atención inspirado en los *transformers* vistos en la sección ???. Esto permite que la red aprenda automáticamente las dependencias espaciales y temporales entre las articulaciones sin necesidad de conocer sus posiciones exactas o conexiones anatómicas previas, además implementan una técnica de separación de datos para captar distintos tipos de movimientos.

#### 3.4.1. Módulos de atención espacial y temporal

Si recordamos en la sección anterior de *transformers* que la entrada de datos es una secuencia bidimensional: una matriz  $X \in \mathbb{R}^{N \times C}$ , donde  $N$  es el número de elementos y  $C$  es el número de canales.

Sin embargo, los datos dinámicos del esqueleto tienen tres dimensiones: **espacio**, **tiempo** y **canales**. Podemos representarlos como un tensor de tercer orden:  $X \in \mathbb{R}^{N \times T \times C}$ , donde  $N$  es el número de articulaciones,  $T$  es el número de fotogramas (frames) y  $C$  son los canales de coordenadas.

La forma ingénua de representar los datos, ya probado por otros autores anteriores, es aplanar el espacio y el tiempo en una sola dimensión  $\hat{N} = NT$ , convirtiendo el tensor de tercer orden en  $X \in \mathbb{R}^{\hat{N} \times C}$ . Este enfoque presentaba varias desventajas, a priori, estaban tratando el espacio y el tiempo como si fueran lo mismo y compartieran estructura, lo cual físicamente no tiene sentido. Por otra parte, el coste computacional de este enfoque es muy alto en tiempo y en memoria.

Este papel propone tres métodos para este desacoplamiento. Los explicaremos brevemente utilizando como ejemplo la atención espacial.

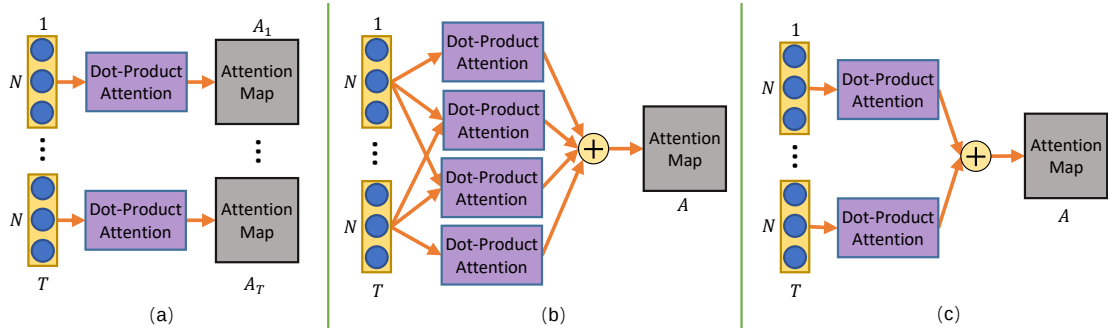


Figura 3.3: Las tres estrategias propuestas en [12]. Grafico sacado de ese papel.



**Estrategia (a)**

Aquí se calcula un mapa de atención único para cada fotograma de manera independiente.

$$Attention_t = softmax(\sigma(X_t)\phi(X_t)'),$$

donde,  $Attention_t \in \mathbb{R}N \times N$  es el mapa de atención para el fotograma  $t$ ;  $X_t \in \mathbb{R}^{N \times C}$  son los datos de ese fotograma; y,  $\sigma$  y  $\phi$  denotan funciones de *embedding*.

Este método no solo tiene un alto coste computacional sino que, además, es incapaz de modelar las relaciones fuera de un mismo fotograma, carece de contexto global.

**Estrategia (b)**

En un giro de 180° los autores proponen otro método. En vez de restringirnos en un fotograma, calculamos las relaciones entre todas las articulaciones a lo largo de todos los fotogramas de forma simultánea. El mapa de atención resultante se comparte para todos los fotogramas.

$$Attention = softmax(\sum_t^T \sum_\tau^T (\sigma(X_t)\phi(X_\tau)')).$$

Aunque es **muy** potente, este método es aún más computacionalmente pesado.

**Estrategia (c)**

c La estrategia que finalmetente propone el papel consiste en solo considerar las articulaciones dentro del mismo fotograma para calcular las relaciones, pero luego sumar y compartir los mapas de atención de todos los fotogramas.

$$Attention = softmax(\sum_t^T (\sigma(X_t)\phi(X_t)')).$$

Si en lugar de tener  $T$  matrices separadas de tamaño  $N \times C$  creamos una nueva matriz  $M$  de tamaño  $N \times TC$  concatenando horizontalmente las matrices de cada fotograma,

$$(X_0 \cdots X_T),$$

por las propiedades de las multiplicaciones de las matrices por bloques, al realizar el producto por su traspuesta obtenemos una matriz cuadrada  $N \times N$ .

Esa única operación produce matemáticamente el mismo resultado exacto que si hubieramos multiplicado y sumado cada fotograma por separado. Pasamos de hacer  $T$  operaciones a hacer una sola operación (que está muy optimizada en las máquinas modernas).

### 3.4.2. Codificación Posicional

Como ocurre en otros modelos de atención pura, las articulaciones del esqueleto se organizan en forma de tensor para poder introducir las en las redes neuronales. Estos tensores carecen de orden o estructura que indique por sí mismo qué representa cada uno (por ejemplo, el índice de la articulación o el índice del fotograma), es necesario usar un módulo de codificación posicional que asigne un identificador único a cada articulación.

Ahora bien, en texto, las palabras van una detrás de otra (1D). Pero los datos del esqueleto tienen dos dimensiones, el espacio (las diferentes articulaciones) y el tiempo (los fotogramas).

Una estrategia ingenua de codificación unificaría estas dimensiones, asignando el valor  $p$  de forma secuencial a través de todo el tensor aplanado. Por ejemplo, con  $N = 3$ , en el fotograma  $t = 1$ , sus posiciones serían 1, 2, 3 y en el fotograma  $t = 2$ , serían 4, 5, 6. El problema de esta topología es que la red no puede distinguir adecuadamente la misma articulación en fotogramas diferentes, ya que la articulación  $n = 1$  recibe representaciones posicionales matemáticamente dispares ( $PE(1)$  vs  $PE(4)$ ).

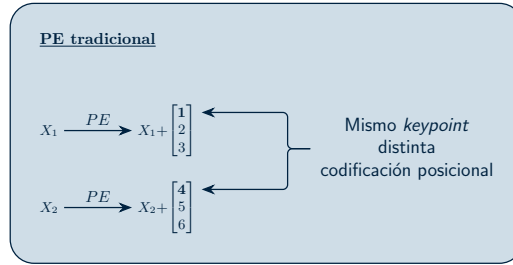


Figura 3.4: Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un **esquema** de lo que ocurre.

Le estamos añadiendo dificultad a la red a la hora de aprender que la articulación 1 y la articulación 4 son, en realidad, la misma parte del cuerpo en diferentes momentos.

Así, para preservar la semántica del esqueleto, los autores proponen separar el proceso en codificación espacial y codificación temporal.

### Codificación de Posición Espacial

Las articulaciones dentro de un mismo fotograma se codifican de forma secuencial, mientras que la misma articulación en fotogramas distintos mantiene exactamente la misma codificación.

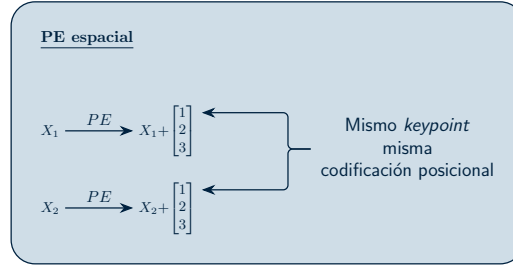


Figura 3.5: Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un **esquema** de lo que ocurre.

Utilizar esta codificación permite que, sin importar el fotograma, una articulación siempre tenga el mismo vector de identidad espacial.

### Codificación de Posición Temporal

Es el proceso análogo e inverso. Las articulaciones dentro de un mismo fotograma se codifican de forma secuencial, mientras que la misma articulación en fotogramas distintos mantiene exactamente la misma codificación.

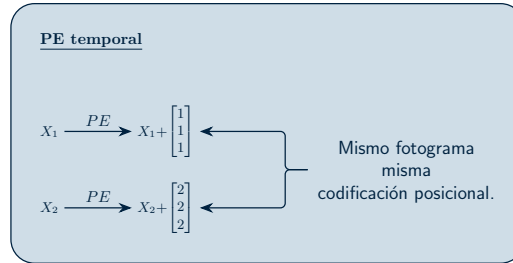


Figura 3.6: Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un **esquema** de lo que ocurre.

#### 3.4.3. Spatial Global Regularization (SGR)

En el procesamiento de imágenes, las redes convolucionales (CNNs) aprovechan que los píxeles cercanos forman bordes y formas, compartiendo pesos locales para encontrar patrones en cualquier parte de la imagen.

En los datos esqueléticos, tenemos un tipo diferente de ventaja, en su mayoría, la anatomía humana es **universal** y **constante**. El *keypoint* que representa la “mano derecha” tiene un significado físico y semántico específico, el cual se mantiene fijo para todos los fotogramas de un video y es consistente a lo largo de todos los sujetos en la base de datos.

Basándose en este conocimiento previo, los autores de la **DSTA** proponen esta regularización para obligar al modelo a aprender atenciones espaciales más generales y aplicables a diferentes muestras.

Para ello, se crea una matriz entrenable o mapa de atención global de tamaño  $N \times N$ , compartida para todas las muestras de datos. Esta matriz actúa como una plantilla que representa los patrones de relaciones que poseen las distintas articulaciones humanas entre sí, que normalmente se mantienen constantes.

Finalmente, este nuevo mapa de atención se combina con el obtenido en la Sección 3.4.1 junto con un factor  $\alpha$  que se encarga de controlar la potencia de esta regularización.

Esta regularización se aplica única y exclusivamente a la dimensión espacial, pues la dimensión temporal no posee esta característica de alineación semántica. Forzar a la red a aprender un patrón unificado global para el tiempo carece de sentido lógico y, como demostraron empíricamente en sus estudios de ablación, perjudicaría el rendimiento del modelo.

El siguiente diagrama muestra la estructura completa del módulo de atención espacial. La atención temporal es análoga salvo por los matices mencionados.

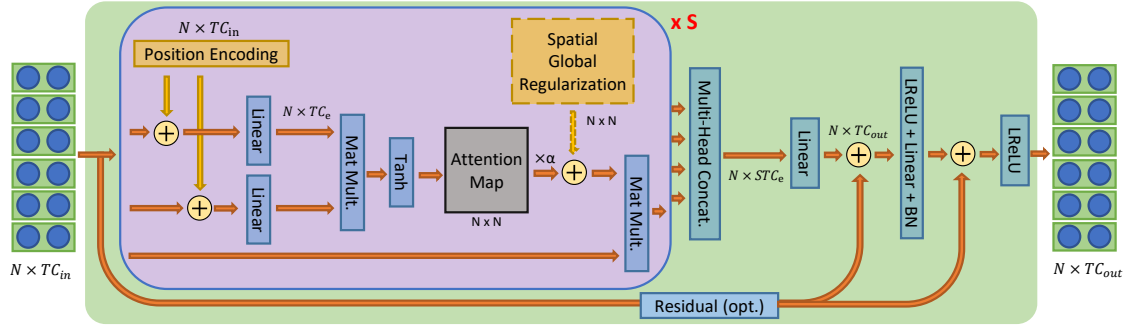


Figura 3.7: Esquema del módulo de atención **espacial**. Obtenido de [12]

Así, en resumen, a la matriz de entrada  $X \in \mathbb{R}^{N \times TC_{in}}$  se le suma la codificación de posición espacial. Luego, se procesa mediante dos funciones de mapeo lineal para proyectarla a un espacio con dimensiones  $N \times TC_e$ . Utilizar un número de canales ( $C_e$ ) menor al de salida ayuda a eliminar la redundancia de características y disminuye el coste computacional de la red.

A continuación, se calcula el mapa de atención utilizando la *estrategia* (c) y se le añade la regularización global espacial. Un detalle muy interesante es el uso de la función de activación *Tanh* en lugar de la habitual *Softmax* para generar este mapa. La razón que dan los autores de este modelo es que la función *Tanh* no restringe los valores a números positivos, lo que otorga al modelo mucha más flexibilidad al permitirle capturar y generar relaciones “negativas” entre las articulaciones. Por último, este mapa de atención resultante se multiplica matricialmente con los datos de la entrada original para obtener las características de salida finales.

Para que el modelo pueda analizar los datos desde múltiples perspectivas simultáneamente, no utiliza una sola cabeza de atención, sino un total de  $S$  cabezas independientes dentro del mismo módulo. Los resultados de todas estas cabezas se concatenan y se procesan a través de una capa lineal para proyectarlos al espacio de salida final  $R^{N \times TC_{out}}$ . Al igual que en el *transformer* original, el proceso termina pasando por una capa *feed-forward* que utiliza *leaky ReLU* como función de activación. Además, se incluyen dos conexiones residuales a lo largo del módulo para estabilizar el entrenamiento de la red e integrar mejor las diferentes características extraídas.

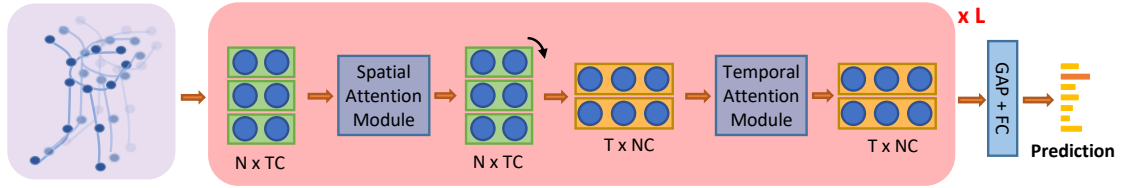


Figura 3.8: Esquema de la red. Obtenido de [12]

La arquitectura **DSTA-Net** organiza este proceso apilando un total de  $L$  capas. En cada una de estas capas, el tensor de entrada se procesa secuencialmente: primero entra al módulo de atención espacial (tratando los datos como  $N$  articulaciones) para aprender la estructura física, y luego la matriz resultante se transpone para entrar al módulo de atención temporal (tratando los datos como  $T$  fotogramas) para aprender el movimiento. Una vez que los datos han pasado por todas las capas, las características finales se reducen mediante un *Global Average Pooling* y se envían a una capa completamente conectada que generaría las puntuaciones finales de clasificación de la acción.

### 3.5. LoRA



# Capítulo 4

---

## Metodología

---

Modelos que vamos a probar, preprocesado de datos (normalización, etc)

### 4.1. Extracción de Keypoints

Como se mencionó, la extracción de keypoints es el proceso mediante el cual se obtiene una representación estructurada del cuerpo humano a partir de imágenes o vídeo, en forma de coordenadas de puntos anatómicos predefinidos, tales como articulaciones, extremidades o rasgos faciales. Esta representación abstrae la información visual relevante para el movimiento y la postura, descartando información irrelevante como el fondo, la iluminación o la apariencia superficial, lo que la convierte en una representación compacta, generalizable y respetuosa con la privacidad, tal y como se argumentó en la sección anterior.

En los últimos años, el avance conjunto de la visión por computador y el aprendizaje profundo ha dado lugar a herramientas de estimación de pose de gran precisión y eficiencia. Entre las más destacadas se encuentran **MediaPipe Holistic**, desarrollada por Google y ampliamente adoptada en la literatura de lengua de signos por su capacidad para detectar simultáneamente cuerpo, manos y rostro en tiempo real con un bajo coste computacional.

**OpenPose**, uno de los primeros sistemas de estimación de pose multi-persona de referencia, ha sido ampliamente utilizado en trabajos previos; sin embargo, presenta limitaciones en términos de eficiencia y robustez frente a enfoques más recientes.

Por otro lado, métodos como **HRNet**, mediante **MMPose**, ofrecen una elevada precisión en la localización de keypoints en imágenes estáticas gracias a su arquitectura de alta resolución, aunque no incorporan de forma explícita mecanismos de modelado temporal, lo que puede generar variabilidad entre frames en secuencias de vídeo. Este es el método que consigue los mejores resultados si la capacidad de cómputo no es una limitación.

Finalmente, **AlphaPose** es un sistema de estimación de pose de alta precisión que destaca especialmente en escenarios multi-persona, condiciones de oclusión y vídeos de

baja calidad, lo cual resulta especialmente relevante para nuestro caso de uso. Además, a diferencia de HRNet en sus configuraciones más pesadas, AlphaPose está optimizado para un mejor compromiso entre precisión y velocidad, permitiendo una extracción más eficiente en entornos prácticos.

#### 4.1.1. Alphapose

Como mencionamos AlphaPose es robusto a condiciones de oclusión y vídeos de baja calidad y es considerablemente menos exigente que otros métodos manteniendo una precisión y velocidad razonables.

AlphaPose, como muchas de estas herramientas, ofrece la capacidad de cambiar el modelo utilizado para la representación del esqueleto, existiendo principalmente dos conjuntos de datos de pose de cuerpo completo:

- **COCO-WholeBody**, una ampliación del conjunto de datos **MS COCO** enfocada en el cuerpo humano completo. Incluye anotaciones detalladas de puntos clave del cuerpo, manos, cara y pies, lo que permite realizar estimaciones de pose más completas y precisas en tareas de análisis del movimiento humano.
- **HALPE-136**, un conjunto de datos más reciente que proporciona una anotación aún más densa (136 puntos clave), con especial énfasis en manos y rostro. Este mayor nivel de detalle puede resultar beneficioso en tareas que requieren capturar movimientos finos, como el reconocimiento de lengua de signos, aunque a costa de una mayor complejidad computacional y posible sensibilidad al ruido en entornos no controlados.

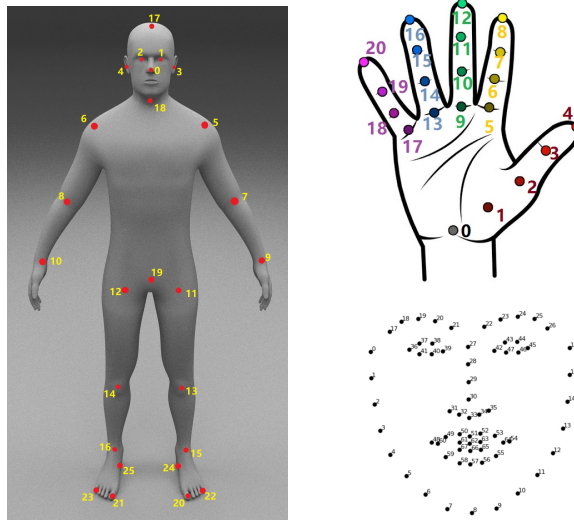


Figura 4.1: Keypoints en el Modelo Halpe-136

AlphaPose devuelve los puntos clave video por video en formato JSON estructurado de la siguiente forma.



```
[
// Primera persona, primera imagen
{
    "image_id" : string, image_1_name,
    "category_id" : int, 1 for person
    "keypoints" : [x1,y1,c1,...,xk,yk,ck],
    "score" : float,
},
// Segunda persona, primera imagen
{
    "image_id" : string, image_1_name,
    "category_id" : int, 1 for person
    "keypoints" : [x1,y1,c1,...,xk,yk,ck],
    "score" : float,
},
...
// Igual para la segunda imagen
{
    "image_id" : string, image_2_name,
    "category_id" : int, 1 for person
    "keypoints" : [x1,y1,c1,...,xk,yk,ck],
    "score" : float,
},
...
]
```

## 4.2. Normalización de los Datos

## 4.3. Multi Stream Keypoint Attention

En esta sección se presentará la arquitectura principal del modelo *Multi-stream Keypoint Attention* (MSKA) detallando el razonamiento detrás de la misma. Es importante notar que este modelo **no** es *end-to-end*, necesita la representación en glosas, se utilizan para asegurar que el modelo realmente aprende el significado detrás de los *keypoints*.

### 4.3.1. Data Augmentation

Como suele ocurrir en aprendizaje profundo, la falta de datos es un muro contra el cual nos estrellamos siempre. Este problema es especialmente cierto en conjuntos de datos cuya anotación manual requiera conocimiento experto como es el caso de la lengua de signos. Aunque ha habido muchos esfuerzos para solventar esta limitaciones, como la generación de glosas de forma automática utilizado, por ejemplo, LLMs [8], este problema no está cerca de ser resuelto. Así, para compensar esta escasez de datos se decide utilizar técnicas de aumento de datos clásicas en problemas que tratan con *keypoints*.

La postura y la ubicación del intérprete varían de forma natural al realizar un signo. Esto permite introducir variabilidad controlada en los datos sin corromper su semántica ni generar muestras inverosímiles. Entre las transformaciones espaciales más comunes y efectivas se encuentran las **rotaciones**, las **traslaciones** y los **cambios de escala (zoom)**.

Sorprendentemente, los autores observaron que la eficacia del modelo comenzó a disminuir específicamente al integrar los métodos de traslación y cambio de escala. Su hipótesis es que estas estrategias de aumento introdujeron discrepancias en la alineación de los datos con el conjunto de validación. En lugar de ayudar a generalizar, estas alteraciones terminaron provocando un sobreajuste. Así, decidieron mantener únicamente las rotaciones.

Además de estos métodos, proponemos dos métodos de aumento de datos y regularización específicos para la tarea de reconocimiento de la pose: **Joint Dropout** y **ruido gaussiano**, su funcionamiento y la motivación de su uso se explicará en las secciones siguientes.

### Rotaciones

El objetivo es que el modelo sea robusto ante pequeñas variaciones en la inclinación o el ángulo del intérprete. Para lograrlo, se aplica una rotación aleatoria de  $\alpha$  grados con una probabilidad  $p$ . Si una matriz  $\mathbf{P} \in \mathbb{R}^{n \times 2}$  contiene las coordenadas  $(x, y)$  de los puntos clave de un fotograma, la transformación se calcula mediante el siguiente producto matricial.

$$\begin{bmatrix} x'_0 & y'_0 \\ x'_1 & y'_1 \\ \vdots & \vdots \\ x'_n & y'_n \end{bmatrix} = \begin{bmatrix} x_0 & y_0 \\ x_1 & y_1 \\ \vdots & \vdots \\ x_n & y_n \end{bmatrix} \cdot \underbrace{\begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}}_{\text{Matriz de rotaciones traspuesta}}$$

**Nota 4.3.1.** Al multiplicar la matriz de coordenadas (donde cada fila es un punto) por la derecha, necesitamos la matriz de rotación traspuesta. La traspuesta de la matriz de rotación estándar para un ángulo  $\alpha$  en sentido antihorario cambia el signo del seno inferior al seno superior. Esto nos permite realizar la rotación del fotograma en un paso.

Es importante acotar el rango de esta transformación; rotaciones extremas (por ejemplo, de  $180^\circ$ ) carecen de sentido práctico, ya que introducen escenarios que el modelo jamás encontrará en un entorno real. Por eso nos limitamos a una rotación de  $\pm 15^\circ$ .

### Aumento Temporal

Las manos ocupan un área reducida del video y son sumamente vulnerables a los movimientos rápidos que ocurren de manera natural durante la comunicación en lenguaje de señas. La idea al alterar aleatoriamente la longitud de la secuencia, es que el modelo se vea forzado a aprender las características de estos movimientos sin importar la velocidad a la que ocurran.

Para modificar la longitud temporal de las secuencias de puntos clave se seleccionan fotogramas válidos de forma completamente aleatoria dentro de un rango determinado. Si seleccionamos una proporción  $< 1$  de fotogramas simulamos a la persona hacer la misma seña, pero a mayor de velocidad. Si la proporción es  $> 1$  se está añadiendo redundancia o manteniendo la información de los fotogramas por más tiempo simulamos a la persona hacer la misma seña pero más lentamente.

A nivel de implementación, la expansión de la secuencia se realiza duplicando fotogramas de forma aleatoria, mientras que su reducción se logra descartándolos, preservando en todo momento el orden cronológico original.

### ***Joint Dropout***

Siguiendo la idea que proponen en *Leveraging Artificial Occluded Samples for Data Augmentation in Human Activity Recognition* [16], buscamos simular la pérdida de información que ocurre cuando una o más de las extremidades del sujeto están ocultas por algún obstáculo o el modelo de extracción de puntos clave es incapaz de extraer la información de algunas extremidades.

El método consiste en eliminar la información de ciertos keypoints agrupándolos por extremidad, por ejemplo *mano derecha*, *mano izquierda*, *brazo izquierdo* y *brazo derecho*. Además los investigadores señalan que la oclusión debe afectar la duración completa de la acción para forzar a la red a aprender de las extremidades restantes.

Se implementa una versión simplificada de la técnica, que trata con datos *2D* a diferencia de los datos *3D* con los que se trata en el papel mencionado. Como detalle, no tiene sentido la oclusión total de las extremidades, ya que esto privaría a la red de características representativas e introduciría ruido perjudicial en el entrenamiento. Así, basándonos en [16] nos limitamos a eliminar 2 extremidades como máximo.

### **Ruido Gaussiano**

Otra situación que ocurre frecuentemente en este tipo de tareas son los micromovimientos de la persona, las interferencias del entorno y los temblores naturales que ocurren al capturar el movimiento. El objetivo principal de implementar el ruido gaussiano es simular estas perturbaciones comunes. Nos basamos en parte en *Enhancing Human Action Recognition with 3D Skeleton Data: A Comprehensive Study of Deep Learning and Data Augmentation* [17].

Se añade ruido que sigue una distribución normal  $\mathcal{N}(\mu, \sigma^2)$ . Para asegurar que el efecto del ruido sea perceptible pero sin llegar a distorsionar gravemente los datos originales, los autores configuraron la media,  $\mu$ , en 0 y la desviación estándar  $\sigma$  en 0,01.

### 4.3.2. Tokenizador

### 4.3.3. Arquitectura del Modelo de Reconocimiento

La lengua de signos no concentra el significado en una sola articulación. Sino que lo distribuye a través de distintos canales simultáneamente: las configuraciones de las manos, los movimientos de los brazos y las expresiones faciales. Para aprovechar esta riqueza de información, el sistema utiliza un mecanismo de autoatención que le permite analizar cada flujo de manera independiente. Posteriormente, combina la información obtenida de cada flujo mediante técnicas de destilación de conocimiento para construir una representación más completa y precisa.

La arquitectura de este módulo, cuyos componentes se detallan en los apartados subsiguientes, se ilustra en el siguiente diagrama.

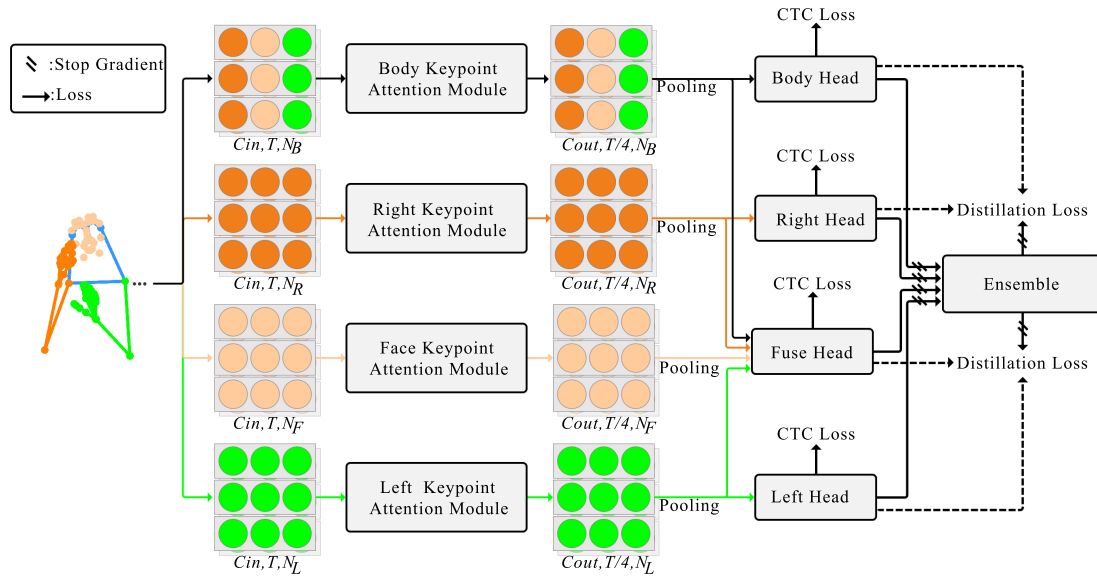


Figura 4.2: Esquema de la estructura del módulo de reconocimiento del modelo. Imagen de [9]

### Desacoplamiento de Keypoints

Como mencionábamos el modelo **MSKA-SLR** deja de utilizar un único flujo de procesamiento (*stream*) y adopta una estrategia de separación de puntos clave inspirada en los mecanismos de percepción humana.

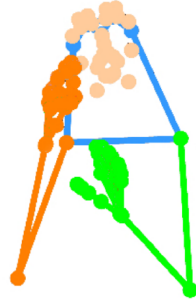


Figura 4.3: Esquema de la división de las extremidades. Imagen de [9]

Como se observa en la figura anterior, la secuencia global extraída del vídeo se segmenta en cuatro flujos (*streams*) independientes: **mano derecha**, **mano izquierda**, **rostro** y **cuerpo entero**.

Este modelado paralelo captura las características específicas de cada región anatómica, mitigando el sesgo de los enfoques monolíticos donde los movimientos amplios del cuerpo pueden eclipsar las variaciones sutiles de los dedos o de la boca, que contienen información muy relevante. La separación asegura que los detalles finos no se pierdan en la representación global.

Además, el procesamiento independiente permite que la red asigne implícitamente la importancia adecuada a cada fuente de información durante la etapa posterior de fusión.

Una vez divididos los flujos, cada subsecuencia se procesa mediante su propio módulo de atención de puntos clave (*Keypoint Attention Module*).

### Módulo de Atención de Keypoints

Este módulo integra los mecanismos de atención espacio-temporal descritos en la Sección 3.4. Los pesos de estos bloques son independientes y no se comparten entre los distintos flujos. El siguiente diagrama ilustra la adaptación de esta estructura a los *keypoints* corporales.

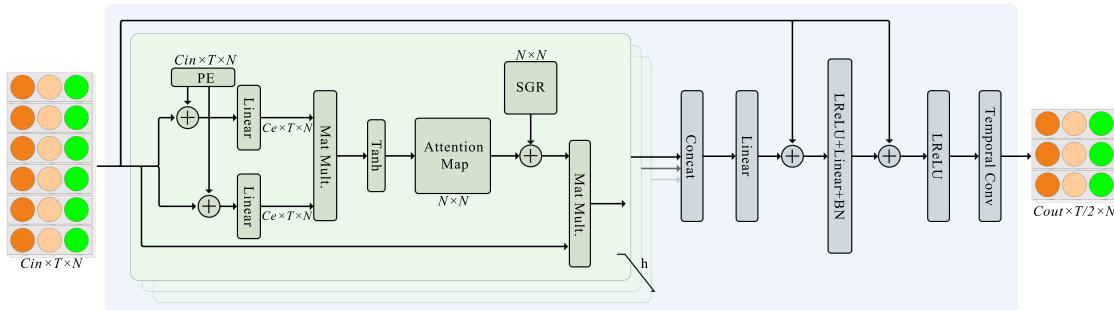


Figura 4.4: Esquema del módulo de atención de *keypoints* del cuerpo. Se toma  $h = 6$ . Imagen de [9]

El flujo de datos comienza con una secuencia de puntos clave estructurada como un tensor multidimensional  $X \in \mathbb{R}^{C \times T \times N}$ , donde:

- $C$  representa los **canales de entrada**, definidos por el vector  $[x_t^n, y_t^n, c_t^n]$ . Las variables  $(x_t^n, y_t^n)$  denotan las coordenadas espaciales bidimensionales, mientras que  $c_t^n$  indica el nivel de confianza asignado por el estimador de pose al punto clave  $n$  en el instante  $t$ .
- $T$  corresponde a la dimensión temporal (número total de fotogramas).
- $N$  es el número total de puntos clave específicos del flujo.

A diferencia de la arquitectura *DSTA* descrita en la *Sección 3.4*, la codificación posicional se restringe a la dimensión espacial, delegando el modelado temporal a convoluciones  $2D$  en etapas posteriores.

Esta modificación evita el incremento de parámetros inherente a la atención temporal. Dado el tamaño reducido de los conjuntos de datos de lenguaje de señas, una alta parametrización elevaría significativamente el riesgo de sobreajuste.

Delegado el modelado temporal a las convoluciones  $2D$ , el tensor se procesa mediante una cascada de ocho bloques de atención independientes por flujo. Cada bloque incorpora una convolución temporal al final de su estructura; no obstante, solo el primer y quinto bloque aplican un paso (*stride*) de 2. Esta configuración contrae la resolución temporal de forma escalonada, reduciendo los fotogramas de  $T$  a  $T/4$  ( $T \rightarrow T/2 \rightarrow T/4$ ). Simultáneamente, la dimensión de los canales de entrada ( $C_{in}$ ) se expande progresivamente (64, 128 y 256) hasta fijar un valor de salida  $C_{out} = 256$ .

Finalizada la cascada de atención, el tensor resultante  $\in \mathbb{R}^{256 \times T/4 \times N}$  es procesado mediante un agrupamiento espacial (*Spatial Pooling*) antes de alimentar a la red de cabecera. Esta operación promedia la dimensión correspondiente a los *keypoints* ( $N$ ), colapsando la representación espacial explícita del esqueleto. Se obtiene así un tensor bidimensional de  $T/4 \times 256$  que sintetiza el espacio en características abstractas, aislando la dinámica temporal para la clasificación final de las glosas.

### Redes de Cabecera (*Head Networks*) y Fusión

Cada flujo de procesamiento (mano izquierda, mano derecha, rostro y cuerpo) tiene su propia red de cabecera independiente. Estas estructuras modelan el contexto temporal final de los datos, extrayendo la dinámica global del movimiento a lo largo del tiempo.

Cada cabeza está compuesta por una capa lineal temporal, una de normalización por lotes (*batch normalization*), una de activación *ReLU* y un bloque convolucional temporal, que contiene dos capas de convolución  $1D$  con un tamaño de kernel de 3. Termina con otra lineal y un último *ReLU*.

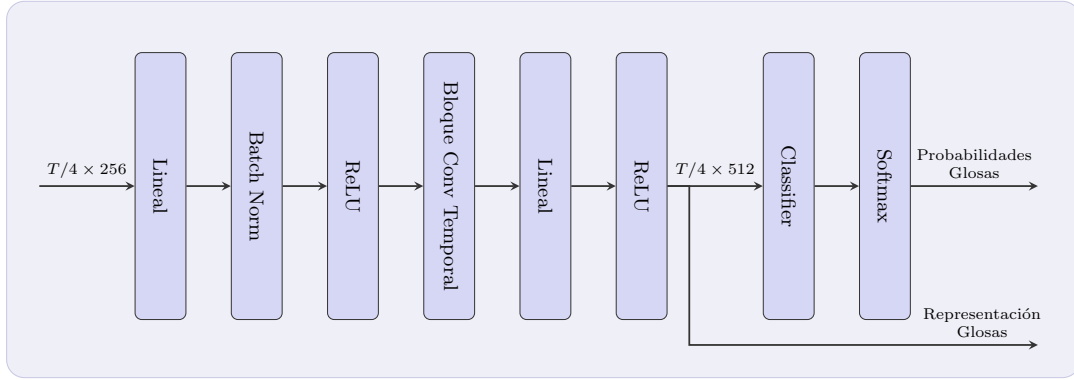


Figura 4.5: Esquema las redes de cabecera.

Produce lo que los autores denominan la **representación de la glosa**, con dimensiones  $T/4 \times 512$ . Finalmente, un clasificador lineal acoplado a una activación *softmax* estima la distribución de probabilidades para predecir la glosa articulada. Esta probabilidad se utiliza principalmente para el cálculo de la pérdida; el modelo de traducción recibe la representación de la glosa.

Cada cabezal se entrena y se corrige utilizando la función de pérdida CTC expuesta en la *Sección 3.3* de manera independiente.

Para aprovechar al máximo la separación de la información espacial, la arquitectura incorpora un cabezal auxiliar denominado **Cabezal de Fusión** (*Fuse Head*). Su objetivo es asimilar e integrar las características extraídas por los cuatro flujos independientes (manos, rostro y cuerpo) antes de emitir una decisión final. La estructura interna de este módulo de fusión es idéntica a la de las redes de cabecera individuales y, de igual manera, es supervisada por su propia función de pérdida CTC ( $\mathcal{L}_{CTC}^{fuse}$ ).

Durante la inferencia, las probabilidades de glosa a nivel de fotograma predichas por los distintos flujos se promedian y se envían a un módulo de *ensemble* para decodificar la secuencia final. Este enfoque unifica las representaciones locales, mejorando significativamente la robustez y precisión de las predicciones.

### Función de Pérdida y Autodestilación

El entrenamiento conjunto de la red *MSKA-SLR* está guiado por dos componentes principales. En primer lugar, se aplica la pérdida CTC a las salidas de cada una de las cuatro cabezas.

$$\mathcal{L}_{CTC} = \mathcal{L}_{CTC}^{left} + \mathcal{L}_{CTC}^{right} + \mathcal{L}_{CTC}^{body} + \mathcal{L}_{CTC}^{fuse}$$

En segundo lugar, para complementar la naturaleza global de la pérdida CTC y lograr una interacción más profunda entre los flujos, el modelo implementa una **autodestilación**

**a nivel de fotograma.** Esta técnica calcula el promedio de las probabilidades emitidas por las cuatro redes de cabecera principales y lo utiliza como un *pseudo-objetivo*.

A través de una pérdida de destilación ( $\mathcal{L}_{Dist}$ ), se minimiza la divergencia KL (*Kullback-Leibler*)<sup>1</sup> entre este pseudo-objetivo unificado y las predicciones individuales de cada flujo.

Para asegurar la estabilidad del entrenamiento, se aplica una operación de **parada de gradiente** sobre el ensamblaje. Al desconectar el *pseudo-objetivo* del grafo computacional durante la retropropagación, se garantiza que la pérdida de destilación actúe únicamente sobre los pesos de los flujos individuales. Esto evita la retroalimentación del error hacia el ensamblaje, previniendo el colapso del modelo y asegurando que el objetivo de referencia actúe como un ancla estática.

Así, la función de pérdida global es la siguiente.

$$\mathcal{L}_{SLR} = \mathcal{L}_{CTC} + \mathcal{L}_{Dist}$$

De este modo, el conocimiento unificado del ensamblaje se transfiere a cada flujo individual. Esto garantiza que cada cabezal no solo aprenda de sus propias características locales, sino que optimice sus predicciones basándose en la comprensión global de toda la red.

#### 4.3.4. Arquitectura del módulo de traducción

Los autores expanden el modelo de reconocimiento hacia un modelo de traducción mediante la adición de una red de traducción externa. La arquitectura sigue un paradigma de codificador-decodificador.

La red MSKA-SLR hace papel de codificador visual (*Visual Encoder*), su salida, la **representación de la glosa**, sirve como el “lenguaje intermedio” que entiende la red.

El responsable de generar la secuencia textual final es el decodificador (*Translation Network*). Exploraremos la arquitectura original y la expandiremos utilizando **modelos de lenguaje**.

#### Integración mediante MLP

Dado que el codificador visual opera en un dominio de características espaciales y temporales comprimidas ( $T/4 \times 512$ ), es necesario adaptar esta salida antes de introducirla en el decodificador de lenguaje.

Para este fin, el sistema emplea un **perceptrón multicapa (MLP)** con dos capas ocultas. Este bloque actúa como un adaptador de dominio muy básico: proyecta las representaciones de la glosa al espacio semántico que el modelo de lenguaje espera, alineando las características extraídas por el flujo de fusión con las entradas del decodificador.

---

<sup>1</sup>La divergencia de Kullback-Leibler (KL), también llamada entropía relativa, es una medida estadística que cuantifica cuánto difiere una distribución de probabilidad de otra [18].



### MBART como decodificador

El artículo opta por utilizar **mBART** (*Multilingual BART*) [19] como la red de traducción principal.

mBART es un modelo de lenguaje preentrenado con una arquitectura tipo *transformer encoder-decoder* que ha demostrado un rendimiento sobresaliente en tareas de traducción automática multilingüe.

Gracias a su preentrenamiento sobre grandes corpus multilingües, el modelo ya codifica las estructuras sintácticas y semánticas necesarias para la decodificación a lenguaje natural, lo que reduce el entrenamiento a alinear las representaciones visuales con su espacio lingüístico.

La pérdida de traducción se calcula mediante entropía cruzada a nivel de token (*token-level cross-entropy loss*): el decodificador predice la secuencia objetivo de forma autorregresiva, optimizando la probabilidad de cada token correcto dado el contexto previo y la representación visual codificada.

### Modelo de lenguaje pequeño como decodificador

Los modelos *decoder-only* modernos han demostrado capacidades de generalización, traducción y comprensión lingüística superiores a muchos modelos *encoder-decoder*, gracias a su preentrenamiento sobre corpus de texto de mayor escala. Así, en lugar de un modelo *encoder-decoder* como mBART, optamos por integrar un modelo de lenguaje con arquitectura exclusivamente de decodificador (*decoder-only*).

En esta arquitectura, las características visuales no pasan por un codificador independiente. En su lugar, se emplea una estrategia de **prefijo visual** (*visual prefix conditioning*), inspirada en modelos multimodales como LLaVA [24]:

1. El *VLMapper* proyecta las características visuales al espacio de embeddings del modelo de lenguaje.
2. Estas características se tratan como una secuencia de tokens continuos y se concatenan al inicio de la entrada.
3. Se anexa un *prompt* textual de instrucción que guía al modelo hacia la tarea de traducción.

La secuencia unificada que procesa el modelo adopta la estructura:

[Tokens Visuales] + [Prompt Textual] + [Traducción Objetivo].

La función de pérdida se aplica únicamente sobre los tokens de la traducción objetivo. De este modo, el modelo no recibe penalización por su comportamiento sobre el contexto de entrada, concentrando toda la señal de supervisión en aprender a generar la traducción correcta condicionada a la información visual.

Dado que un ajuste fino completo de un modelo de lenguaje podría desestabilizar sus capacidades lingüísticas preentrenadas y exigiría recursos computacionales prohibitivos, la red de traducción se adapta mediante **LoRA** (explicado en *Sección ??*).

Antes de explorar dicha estrategia, evaluaremos su capacidad en un escenario *zero-shot*, siguiendo la línea del trabajo **Spotter+GPT** [?] presentado en el estado del arte. En dicho trabajo, las glosas reconocidas se pasan directamente a un LLM mediante un *prompt* de instrucción, sin ningún tipo de entrenamiento adicional, obteniendo traducciones coherentes gracias a la capacidad lingüística preentrenada del modelo.

# Capítulo 5

---

## Experimentación y análisis de resultados

---

### 5.1. Keypoints

### 5.2. MSKA

#### 5.2.1. Recreando el papel original

#### 5.2.2. Utilizando un SLM

Usaremos modelos de la familia de modelos de pesos abiertos **Qwen** [21] como red de traducción.

Qwen2.5 han sido preentrenados sobre corpus de hasta 36 billones de tokens, con una composición diversa y de alta calidad que incluye texto web, libros, código, datos STEM, razonamiento lógico, contenido multilingüe y datos sintéticos [21]. Este volumen y diversidad de preentrenamiento dota a los modelos de una comprensión lingüística profunda especialmente útil para formar frases coherentes a partir de glosas que carecen de corrección gramatical. Además, la familia Qwen ofrece modelos de tamaño variado con un rendimiento competitivo frente a modelos mayores de otras familias, lo que los hace idóneos para entornos con restricciones de memoria de GPU.

Concretamente, trataremos con **Qwen2.5-1.5B**, **Qwen2.5-7B** y **Qwen2.5-14B** [22].

MSKA-SLR como Spotter + Qwen

MSKA-SLR+Qwen (FineTunning y LoRA)

### 5.3. Comparativa con state of the art

Un adendum

## Capítulo 6

---

### Conclusión y trabajo futuro

---



## Bibliografía

---

- [1] NADA E. ELSHAMI, AHMAD SALAH, AMR ABDELLATIF Y HEBA MOHSEN, *Toward Robust Human Pose Estimation Under Real-World Image Degradations and Restoration Scenarios*, Information, vol. 16, no. 11, art. 970, 2025. Disponible en: <https://www.mdpi.com/2078-2489/16/11/970>
- [2] HAO-SHU FANG, JIEFENG LI, HONGYANG TANG, CHAO XU, HAOYI ZHU, YULIANG XIU, YONG-LU LI Y CEWU LU, *AlphaPose: Whole-Body Regional Multi-Person Pose Estimation and Tracking in Real-Time*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022. Disponible en: <https://github.com/Fang-Haoshu/Halpe-FullBody>
- [3] ASHISH VASWANI, NOAM SHAZEER, NIKI PARMAR, JAKOB USZKOREIT, LLION JONES, AIDAN N. GOMEZ, LUKASZ KAISER E ILLIA POLOSUKHIN, *Attention Is All You Need*, arXiv preprint, art. 1706.03762, 2023. Disponible en: <https://arxiv.org/abs/1706.03762>
- [4] HAO-SHU FANG, JIEFENG LI, HONGYANG TANG, CHAO XU, HAOYI ZHU, YULIANG XIU, YONG-LU LI Y CEWU LU, *AlphaPose: Whole-Body Regional Multi-Person Pose Estimation and Tracking in Real-Time*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022.
- [5] HAO-SHU FANG, SHUQIN XIE, YU-WING TAI Y CEWU LU, *RMPE: Regional Multi-person Pose Estimation*, Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017.
- [6] JIEFENG LI, CAN WANG, HAO ZHU, YIHUAN MAO, HAO-SHU FANG Y CEWU LU, *Crowdpose: Efficient crowded scenes pose estimation and a new benchmark*, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10863-10872, 2019.
- [7] HAO ZHOU, WENGANG ZHOU, YUN ZHOU Y HOUQIANG LI, *Spatial-Temporal Multi-Cue Network for Continuous Sign Language Recognition*, CoRR, vol. abs/2002.03187, 2020. Disponible en: <https://arxiv.org/abs/2002.03187>
- [8] JIANYUAN GUO, PEIKE LI Y TREVOR COHN, *Bridging Sign and Spoken Languages: Pseudo Gloss Generation for Sign Language Translation*, arXiv preprint, art. 2505.15438, 2025. Disponible en: <https://arxiv.org/abs/2505.15438>

- [9] MO GUAN, YAN WANG, GUANGKUN MA, JIARUI LIU Y MINGZU SUN, *Multi-Stream Keypoint Attention Network for Sign Language Recognition and Translation*, arXiv preprint, art. 2405.05672, 2024. Disponible en: <https://arxiv.org/abs/2405.05672>
- [10] OZGE MERCANOGLU SINCAN Y RICHARD BOWDEN, *Spotter+GPT: Turning Sign Spottings into Sentences with LLMs*, Adjunct Proceedings of the 25th ACM International Conference on Intelligent Virtual Agents, pp. 1-6, 2025. Disponible en: <http://dx.doi.org/10.1145/3742886.3756708>
- [11] MATT POST, *A Call for Clarity in Reporting BLEU Scores*, arXiv preprint, art. 1804.08771, 2018. Disponible en: <https://arxiv.org/abs/1804.08771>
- [12] LEI SHI, YIFAN ZHANG, JIAN CHENG Y HANQING LU, *Decoupled Spatial-Temporal Attention Network for Skeleton-Based Action Recognition*, arXiv preprint, art. 2007.03263, 2020. Disponible en: <https://arxiv.org/abs/2007.03263>
- [13] KISHORE PAPINENI, SALIM ROUKOS, TODD WARD Y WEI-JING ZHU, *Bleu: a Method for Automatic Evaluation of Machine Translation*, Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311-318, 2002. Disponible en: <https://aclanthology.org/P02-1040/>
- [14] CHIN-YEW LIN, *ROUGE: A Package for Automatic Evaluation of Summaries*, Text Summarization Branches Out, pp. 74-81, 2004. Disponible en: <https://aclanthology.org/W04-1013/>
- [15] ALEX GRAVES, SANTIAGO FERNÁNDEZ, FAUSTINO GOMEZ Y JÜRGEN SCHMIDHUBER, *Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks*, ICML 2006 - Proceedings of the 23rd International Conference on Machine Learning, vol. 2006, pp. 369-376, 2006. Disponible en: <https://doi.org/10.1145/1143844.1143891>
- [16] EIRINI MATHE, IOANNIS VERNIKOS, EVAGGELOS SPYROU Y PHIVOS MYLONAS, *Leveraging Artificial Occluded Samples for Data Augmentation in Human Activity Recognition*, Sensors, vol. 25, no. 4, art. 1163, 2025. Disponible en: <https://www.mdpi.com/1424-8220/25/4/1163>
- [17] CHU XIN, SEOKHWAN KIM, YONGJOO CHO Y PARK KYOUNG SHIN, *Enhancing Human Action Recognition with 3D Skeleton Data: A Comprehensive Study of Deep Learning and Data Augmentation*, Electronics, vol. 13, no. 4, art. 747, 2024. Disponible en: <https://www.mdpi.com/2079-9292/13/4/747>
- [18] DIAKO RUDD, *Understanding Kullback-Leibler (KL) Divergence*, Medium, 2024. Disponible en: <https://medium.com/@diako.rudd/understanding-kullback-leibler-kl-divergence-d77c976527c8>
- [19] YINHAN LIU, JIATAO GU, NAMAN GOYAL, XIAN LI, SERGEY EDUNOV, MARJAN GHAZVININEJAD, MIKE LEWIS Y LUKE ZETTLEMOYER, *Multilingual Denoising Pre-training for Neural Machine Translation*, CoRR, vol. abs/2001.08210, 2020. Disponible en: <https://arxiv.org/abs/2001.08210>



- [20] ABHINAV P. M., SUJAYKUMAR REDDY M Y OSWALD CHRISTOPHER, *Machine Translation with Large Language Models: Decoder Only vs. Encoder-Decoder*, arXiv preprint, art. 2409.13747, 2024. Disponible en: <https://arxiv.org/abs/2409.13747>
- [21] QWEN, AN YANG, BAOSONG YANG, BEICHEN ZHANG, BINYUAN HUI, BO ZHENG, BOWEN YU, CHENGYUAN LI, DAYIHENG LIU, FEI HUANG, HAORAN WEI, HUAN LIN, JIAN YANG, JIANHONG TU, JIANWEI ZHANG, JIANXIN YANG, JIAXI YANG, JINGREN ZHOU, JUNYANG LIN, KAI DANG, KEMING LU, KEQIN BAO, KEXIN YANG, LE YU, MEI LI, MINGFENG XUE, PEI ZHANG, QIN ZHU, RUI MEN, RUNJI LIN, TIANHAO LI, TIANYI TANG, TINGYU XIA, XINGZHANG REN, XUANCHENG REN, YANG FAN, YANG SU, YICHANG ZHANG, YU WAN, YUQIONG LIU, ZEYU CUI, ZHENRU ZHANG Y ZIHAN QIU, *Qwen2.5 Technical Report*, arXiv preprint, art. 2412.15115, 2025. Disponible en: <https://arxiv.org/abs/2412.15115>
- [22] QWEN TEAM, *Qwen2.5: A Party of Foundation Models*, Qwen Blog, 2024. Disponible en: <https://qwenlm.github.io/blog/qwen2.5/>
- [23] EDWARD J. HU, YELONG SHEN, PHILLIP WALLIS, ZEYUAN ALLEN-ZHU, YUANZHI LI, SHEAN WANG Y WEIZHU CHEN, *LoRA: Low-Rank Adaptation of Large Language Models*, CoRR, vol. abs/2106.09685, 2021. Disponible en: <https://arxiv.org/abs/2106.09685>
- [24] HAOTIAN LIU, CHUNYUAN LI, QINGYANG WU Y YONG JAE LEE, *Visual Instruction Tuning*, arXiv preprint, art. 2304.08485, 2023. Disponible en: <https://arxiv.org/abs/2304.08485>